

Training set optimization

Plan

1. Background / idea
2. Related work
3. Models
4. Data Shapley
5. Tracln
6. Simpler methods
7. Next steps / ideas

Setup

```
In [2]: from pathlib import Path
import sys

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import fitz

plt.style.use("seaborn-v0_8-whitegrid")
pd.set_option("display.max_columns", 120)

# Resolve project root robustly whether notebook cwd is repo root or
notebooks/
cwd = Path.cwd().resolve()
root_candidates = [cwd, cwd.parent, cwd.parent.parent]
PROJECT_ROOT = next((p for p in root_candidates if (p / "src").exists()),
cwd)
OUTPUTS = PROJECT_ROOT / "outputs"

# Ensure imports like `from src...` work in notebook cells
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

print("cwd:", cwd)
print("project root:", PROJECT_ROOT)
print("src exists:", (PROJECT_ROOT / "src").exists())
print("outputs exists:", OUTPUTS.exists())
```

```
cwd: C:\Users\Simen\OneDrive -
NTNU\FYSMAT\INDMAT\MASTER\Master_thesis\notebooks
project root: C:\Users\Simen\OneDrive -
NTNU\FYSMAT\INDMAT\MASTER\Master_thesis
src exists: True
outputs exists: True
```

```

In [3]: def load_if_exists(path: Path) -> pd.DataFrame | None:
        if not path.exists():
            print(f"Missing: {path}")
            return None
        df = pd.read_csv(path)
        print(f"Loaded {path} -> {df.shape[0]} rows x {df.shape[1]} cols")
        return df

def summarize_perf(df: pd.DataFrame, metric: str = "corr_eval") ->
pd.DataFrame:
    cols = [c for c in ["method", "selection_method", "target_island",
    "n_individuals", metric] if c in df.columns]
    if metric not in df.columns:
        raise ValueError(f"Metric '{metric}' not in dataframe")
    out = (
        df[cols]
        .groupby([c for c in cols if c != metric], dropna=False)[metric]
        .agg(["mean", "std", "count"])
        .reset_index()
        .sort_values("mean", ascending=False)
    )
    return out

def display_pdf(
    pdf_files: list[Path],
    page_nums: list[int] | None = None,
    zoom: float = 2.8,
    max_papers: int = 6,
    col_width: float = 7.0,
) -> None:
    import fitz # PyMuPDF

    show_files = pdf_files[:max_papers]
    if len(show_files) == 0:
        print("No pdf files provided")
        return

    if page_nums is None:
        page_nums = [0] * len(show_files)
    elif len(page_nums) < len(show_files):
        page_nums = page_nums + [0] * (len(show_files) - len(page_nums))

    cols = min(len(show_files), 3)
    rows = int(np.ceil(len(show_files) / cols))

    fig, axes = plt.subplots(
        rows,
        cols,
        figsize=(col_width * cols, col_width * 1.35 * rows),
        dpi=250,
    )

```

```

axes = np.array(axes).reshape(-1)

for ax, pdf_path, page_num in zip(axes, show_files, page_nums):
    try:
        with fitz.open(pdf_path) as doc:
            page_idx = int(np.clip(page_num, 0, max(len(doc) - 1, 0)))
            page = doc.load_page(page_idx)
            pix = page.get_pixmap(matrix=fitz.Matrix(zoom, zoom),
alpha=False)

            img = np.frombuffer(pix.samples,
dtype=np.uint8).reshape(pix.height, pix.width, pix.n)
            if img.shape[2] > 3:
                img = img[:, :, :3]

            ax.imshow(img, interpolation="nearest")
            ax.set_title(f"{pdf_path.stem}\npage {page_idx + 1}",
fontsize=10)
            ax.axis("off")
        except Exception as exc:
            ax.text(0.5, 0.5, f"Failed to render\n{pdf_path.name}\n{exc}",
ha="center", va="center")
            ax.axis("off")

    for ax in axes[len(show_files):]:
        ax.axis("off")

plt.tight_layout()
plt.show()

def display_figure(filename: Path) -> None:
    img = plt.imread(filename)
    plt.figure(figsize=(img.shape[1] / 100, img.shape[0] / 100), dpi=100)
    plt.imshow(img)
    plt.axis("off")
    plt.tight_layout()
    plt.show()

```

1) Background / idea

- Ultimate goal: Improve genomic prediction
- Kenneths paper: Worse across population

IDEA

For certain islands, maybe some training individuals do more harm than good. Can we improve prediction on target islands by "optimizing" the training set?

2) Related work

Kenneth

```
In [4]: display_pdf(  
        [PROJECT_ROOT / "papers" / "Aase_2025 - How accurate is genomic.pdf"],  
        [10],  
        zoom=3.2,  
        col_width=8.0,  
        )
```

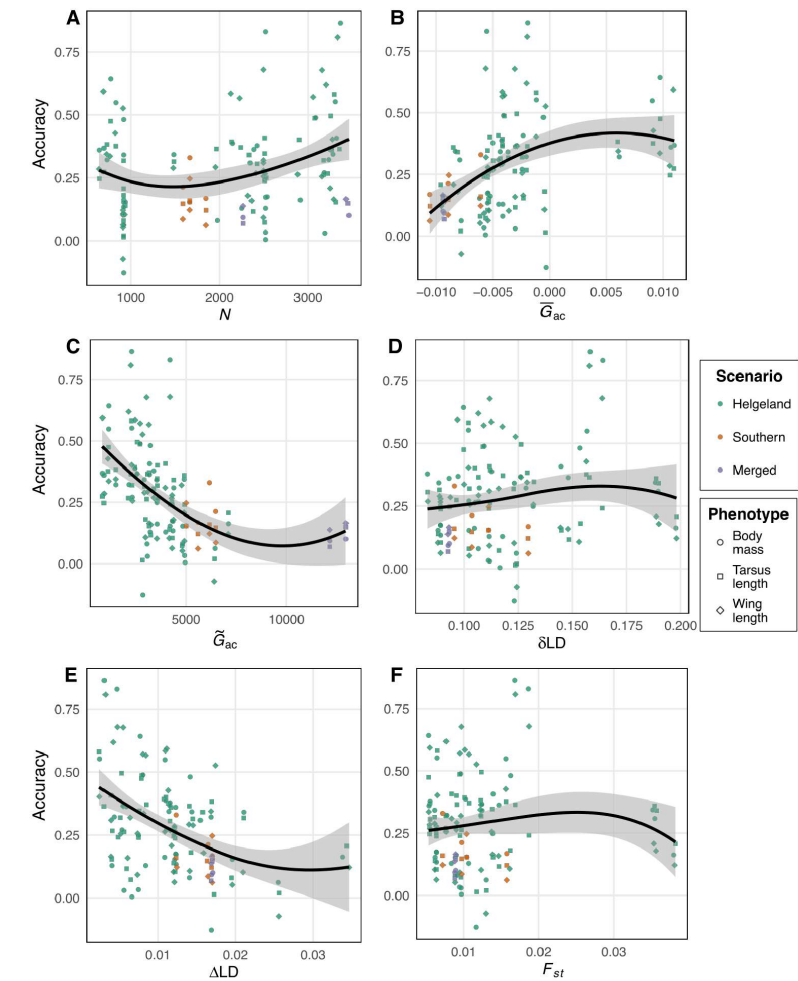


Figure 4. Scatter plots of across-population prediction accuracies for all scenarios and traits, plotted against various statistics. The statistics on the x-axis correspond to the number of individuals N in the training set (A), the mean relatedness $\bar{G}_{ac}$ between individuals in the training and test set (B), the precision $\bar{G}_{ac}$ of relatedness between individuals in the training and test set (C), δLD , a measure of LD-differentiation between the populations (D), ΔLD , a measure of population structure between the training and test set (E), and the population-pairwise fixation index F_{st} (F). The smoothing lines are produced with local regression.

Data shapley

```
In [5]: display_pdf(  
    [  
        PROJECT_ROOT / "papers" / "Ghorbani_2019 - What is your data  
worth.pdf"  
    ],  
    [0],  
    zoom=3.2,  
    col_width=7.5,  
)  
display_figure(PROJECT_ROOT / "papers" / "images" / "Ghorbani_2019 -  
Results.png")
```

What is your data worth? Equitable Valuation of Data

Amirata Ghorbani¹ and James Y. Zou^{2,3,*}

¹Department Electrical Engineering, Stanford University, CA, USA

²Department of Bioemical Data Science, Stanford University, CA, USA

³Chan—Zuckerberg Biohub, San Francisco, CA, USA

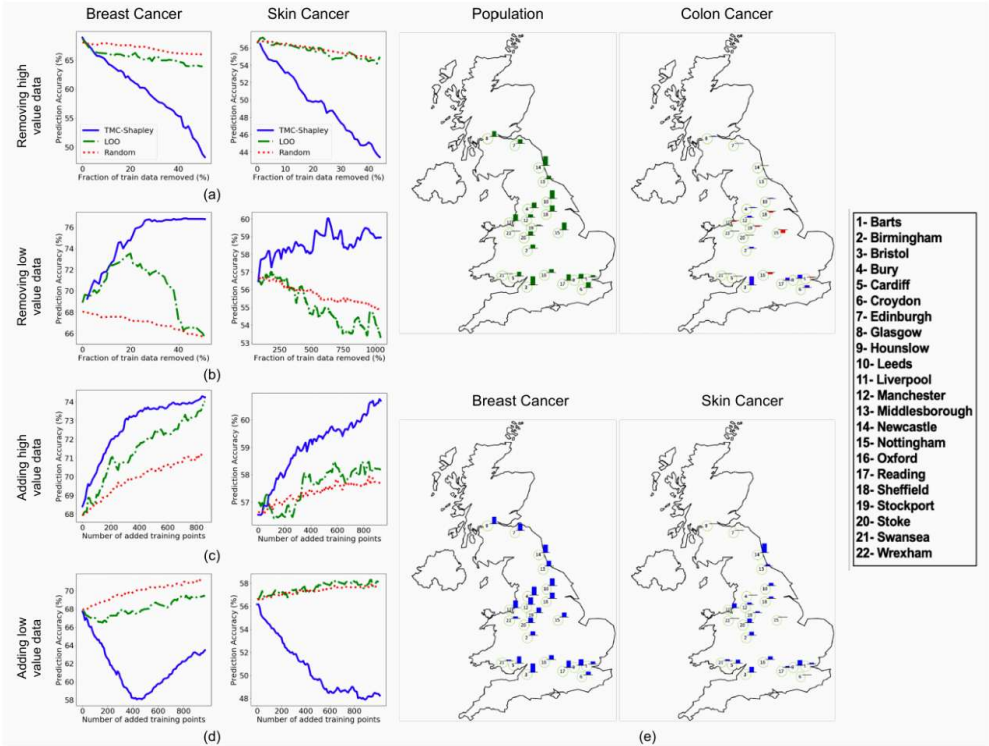
*corresponding author: jamesz@stanford.edu

Abstract

As data becomes the fuel driving technological and economic growth, a fundamental challenge is how to quantify the value of data in algorithmic predictions and decisions. For example, in healthcare and consumer markets, it has been suggested that individuals should be compensated for the data that they generate, but it is not clear what is an equitable valuation for individual data. In this work, we develop a principled framework to address data valuation in the context of supervised machine learning. Given a learning algorithm trained on n data points to produce a predictor, we propose data Shapley as a metric to quantify the value of each training datum to the predictor performance. Data shapley value uniquely satisfies several natural properties of equitable data valuation. We develop Monte Carlo and gradient-based methods to efficiently estimate data Shapley values in practical settings where complex learning algorithms, including neural networks, are trained on large datasets. In addition to being equitable, extensive experiments across biomedical, image and synthetic data demonstrate that data Shapley has several other benefits: 1) it is more powerful than the popular leave-one-out or leverage score in providing insight on what data is more valuable for a given learning task; 2) low Shapley value data effectively capture outliers and corruptions; 3) high Shapley value data inform what type of new data to acquire to improve the predictor.

Introduction Data is the fuel powering artificial intelligence and therefore it has value. Various sectors such as health-care and advertising are becoming more and more dependent on the data generated by individuals; a dependence similar to how they depend on labor and capital¹. As the legal system moves toward recognizing individual data as property², a natural problem to solve is to equitably assign value to this property. The result is the ability to have fair compensation individuals' data. "DATA SHAPLEY" is designed to provide such valuation in one of the widely-used settings of data usage, namely machine learning.

Throughout this work, we focus on valuation of data in the setting of supervised learning as one of the main pillars of artificial intelligence. In supervised machine learning, the data is in the form of input-output pairs; e.g. location and temperature, customers search history and their shopping list, an object's image and its name, and so forth. Using a learning algorithm, a predictive model is then trained to predict the output given the input. The learning algorithm takes a set of train data points (n input-output pairs $\{(x_i, y_i)\}_{i=1}^n$) as its input and outputs the learned predictive model. To assess the quality of the trained model, a performance metric identifies how useful the trained model is. For instance, given a set of input-output pairs of data different from that of the training set, what percentage of outputs are correctly predicted given the inputs. As mentioned, supervised machine learning consists of three ingredients" training data, learning algorithm, and performance metric. As a consequence, to make sense of value for data, we need the same ingredients in our investigation: we are given the fixed data set used for training the machine learning model, the algorithm that trained the model, and the metric to assess the learned model's performance.



TracIn

```
In [6]: display_pdf([PROJECT_ROOT / "papers" / "Garima_2020 - Tracing Gradient
Descent.pdf"], [0], zoom=3.2, col_width=7.5)
```

Garima_2020 - Tracing Gradient Descent
page 1

Estimating Training Data Influence by Tracing Gradient Descent

Garima*
Google
pruthi@google.com

Frederick Liu*
Google
frederickliu@google.com

Satyen Kale
Google
satyenkale@google.com

Mukund Sundararajan[†]
Google
mukunds@google.com

Abstract

We introduce a method called TracIn that computes the influence of a training example on a prediction made by the model. The idea is to trace how the loss on the test point changes during the training process whenever the training example of interest was utilized. We provide a scalable implementation of TracIn via: (a) a first-order gradient approximation to the exact computation, (b) saved checkpoints of standard training procedures, and (c) cherry-picking layers of a deep neural network. In contrast with previously proposed methods, TracIn is *simple* to implement; all it needs is the ability to work with gradients, checkpoints, and loss functions. The method is *general*. It applies to any machine learning model trained using stochastic gradient descent or a variant of it, agnostic of architecture, domain and task. We expect the method to be widely useful within processes that study and improve training data. Code is available at [1].

1 Motivation

Deep learning has been used to solve a variety of real-world problems. A common form of machine learning is supervised learning, where the model is trained on *labelled* data. Controlling the training data input to the model is one of the main quality knobs to improve the quality of the deep learning model. For instance, such a technique could be used to identify and fix mislabelled data using the workflow described in Section 4.1. Our main motivation is to identify practical techniques to improve the analysis of the training data. Specifically, we study the problem of *identifying the influence of training examples on the prediction of a test example*. We propose a method called TracIn for computing this influence, provide a scalable implementation, and evaluate the method experimentally.

2 Related Work

[2, 3] tackle influential training examples in the context of deep learning. We discuss these methods in detail in Section 4.4.

There are related notions of influence used to explain deep learning models that differ in either the target of the explanation or the choice of influencer or both. For instance, [4, 5, 6] identify the influence of features on an individual prediction. [7, 8] identify the influence of features on the

*Equal contribution.

[†]Corresponding author.

Traditional methods

```
In [7]: display_pdf([PROJECT_ROOT / "papers" / "Rincent_2012 - Maximizing the Reliability.pdf",
PROJECT_ROOT / "papers" / "Akdemir_2015 - Optimization of genomic selection.pdf"],
[0, 0], zoom=3.2, col_width=7.5)
```



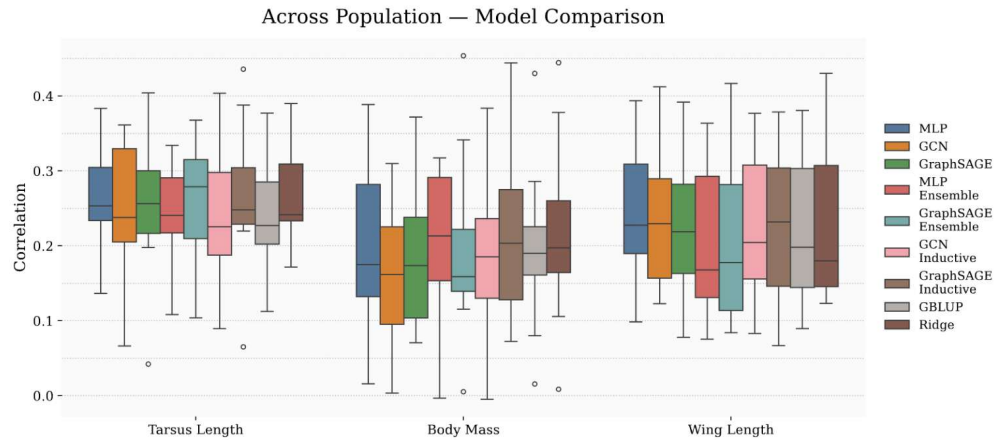
Ideas

- PEV (Predicted error variance). Minimize expected PEV.
- CD (Coefficient of determination). Maximize expected CD.

Models

- 2-step model with ridge regression
- shrinkage fixed

```
In [8]: display_figure(PROJECT_ROOT / "figures" / "loio_single_panel.png")
```



3) Data shapley

Ghorbani, Zou 2019 - What is your data worth? Equitable Valuation of Data

Shapley Value (data valuation)

- $D = \{x_i, y_i\}_{i=1}^n$: full training set (all data sources).
- i : the data source/individual we are valuing.
- $S \subseteq D \setminus \{i\}$: a subset that does not contain i .
- $V(S)$: predictive performance when training on subset S .
- ϕ_i : Shapley value for source i (its average contribution to performance).
- C : normalization constant so contributions are averaged over subset sizes.

\$\$

$$\phi_i = C \sum_{S \subseteq D \setminus \{i\}} \frac{V(S \cup \{i\}) - V(S)}{\binom{n-1}{|S|}}$$

\$\$

Interpretation: for each possible subset S , measure the marginal gain from adding i and average those gains fairly over all subset sizes.

```
In [9]: display_figure(PROJECT_ROOT / "papers" / "images" / "shapley_alg.png")
```

Algorithm 1 Truncated Monte Carlo Shapley

Input: Train data $D = \{1, \dots, n\}$, learning algorithm $\mathcal{A}$, performance score V

Output: Shapley value of training points: $\phi_1, \dots, \phi_n$

Initialize $\phi_i = 0$ for $i = 1, \dots, n$ and $t = 0$

while Convergence criteria not met **do**

$t \leftarrow t + 1$

π^t : Random permutation of train data points

$v_0^t \leftarrow V(\emptyset, \mathcal{A})$

for $j \in \{1, \dots, n\}$ **do**

if $|V(D) - v_{j-1}^t| < \text{Performance Tolerance}$ **then**

$v_j^t = v_{j-1}^t$

else

$v_j^t \leftarrow V(\{\pi^t[1], \dots, \pi^t[j]\}, \mathcal{A})$

end if

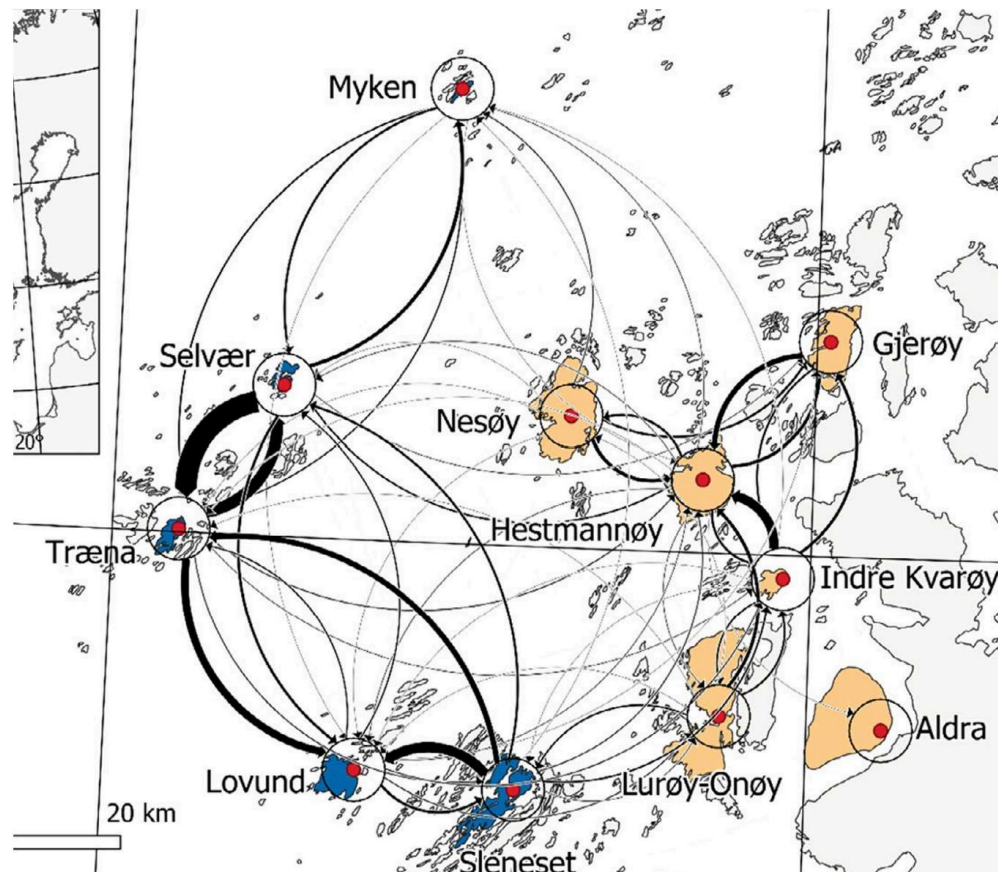
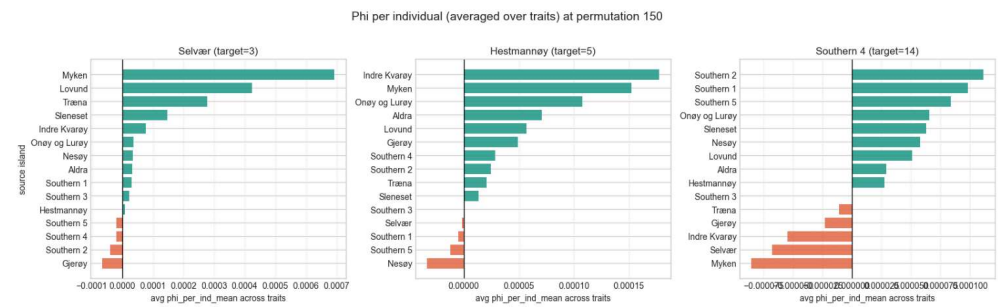
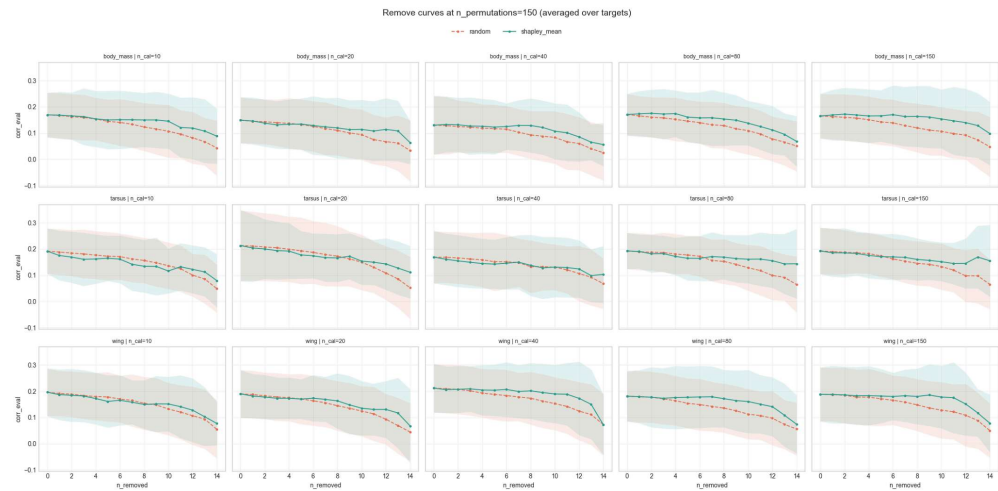
$\phi_{\pi^t[j]} \leftarrow \frac{t-1}{t} \phi_{\pi^{t-1}[j]} + \frac{1}{t} (v_j^t - v_{j-1}^t)$

end for

end while

Will be super slow with 3'000 - 6'000 training individuals. Idea: Do it island-wise.

```
In [10]: display_figure(PROJECT_ROOT / "figures" / "shap_removed_vs_accuracy.png")
display_figure(PROJECT_ROOT / "figures" / "shap_shapvals.png")
display_figure(PROJECT_ROOT / "figures" / "islands.png")
```



4) TracIn

Pruthi et al. 2020 - Estimating Training Data Influence by Tracing Gradient Descent

Main idea

- Goal: estimate how much each training point influences a test prediction.
- Influence is approximated by tracing training checkpoints and accumulating gradient similarity.
- If a train point gradient aligns with the test gradient, it is helpful; opposite direction suggests harmful influence.

A common TracIn score form is:

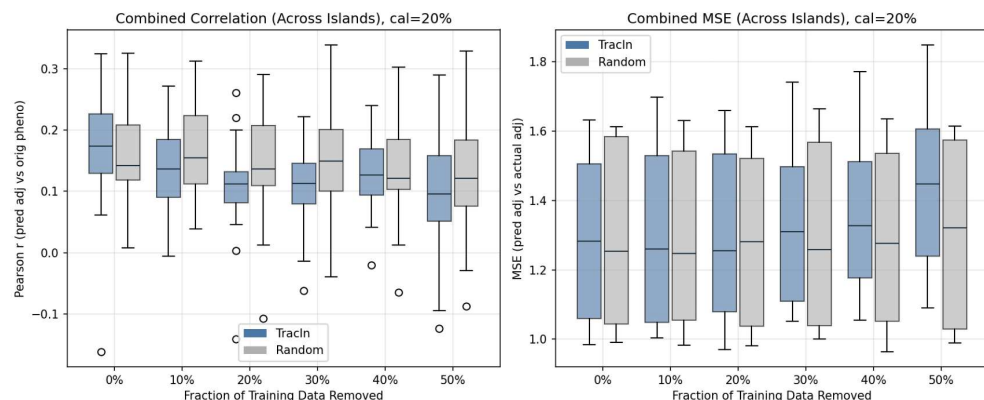
\$\$

$$\mathrm{TracIn}(z_{\mathrm{train}}, z_{\mathrm{test}}) = \sum_t \eta_t \langle \nabla_{\theta} \ell(z_{\mathrm{train}}, \theta_t), \nabla_{\theta} \ell(z_{\mathrm{test}}, \theta_t) \rangle$$

\$\$

where t indexes training checkpoints and η_t is the step size at checkpoint t .

```
In [20]: display_figure(PROJECT_ROOT / "outputs" / "tracin_multi" /
    "combined_removal_curves_cal_20.png")
```



Tracin score distribution not clear, as opposed to Data Shapley.

5) Simpler methods

Methods that do not require "cheat data". They use genotypes (SNP-derived structure) rather than phenotypes for ranking and subset selection.

5.1) Prediction error variance (PEVmean) with GA

- Start from prediction error variance of target breeding values:

$$\mathrm{PEV}_T(S) = \mathrm{Var}(\mathbf{u}_T - \hat{\mathbf{u}}_T \mid S)$$

- Here S is the selected training subset (size k), and T is the target set we want to predict well.

- Under ridge/GBLUP mixed-model assumptions, this covariance is approximated by

$$\mathrm{PEV}_T(S) = \mathbf{G}_{TT} - \mathbf{G}_{TS}(\mathbf{G}_{SS} + \lambda \mathbf{I})^{-1} \mathbf{G}_{ST}, \quad \lambda = \frac{\sigma_e^2}{\sigma_u^2}$$

- Subscript meaning (important):

- $\mathbf{G}_{SS}$: rows/cols both from selected training subset S (within-training block).
- $\mathbf{G}_{TT}$: rows/cols both from target set T (within-target block).
- $\mathbf{G}_{TS}$: rows from T , cols from S (target-to-training relationships).
- $\mathbf{G}_{ST}$: transpose block, rows from S , cols from T .

- Scalar objective (PEVmean) is the average target uncertainty:

$$J(S) = \frac{1}{|T|} \mathrm{tr}(\mathrm{PEV}_T(S)), \quad |S|=k$$

- Intuition: lower $J(S)$ means more reliable predictions for target individuals.

- Exact subset search is infeasible, so we minimize $J(S)$ approximately with a genetic algorithm (selection/crossover/mutation).

5.2) Average GRM ($\bar{G}$) + top- k

- Intuition: rank candidates by genetic relatedness to the target island, then pick the closest.
- Let T be target individuals and c one candidate from source pool.
- Candidate score is mean relationship to target set:

$$s(c) = \frac{1}{|T|} \sum_{t \in T} G_{t,c}$$

where G is the genomic relationship matrix.

- Rank candidates by $s(c)$ in descending order and choose top- k .

5.3) PC distance + top- k

- Fit PCA on candidate training genotypes and keep first m components.
- Let $z_i \in \mathbb{R}^m$ be PC scores for individual i .
- Compute target centroid in PC space:

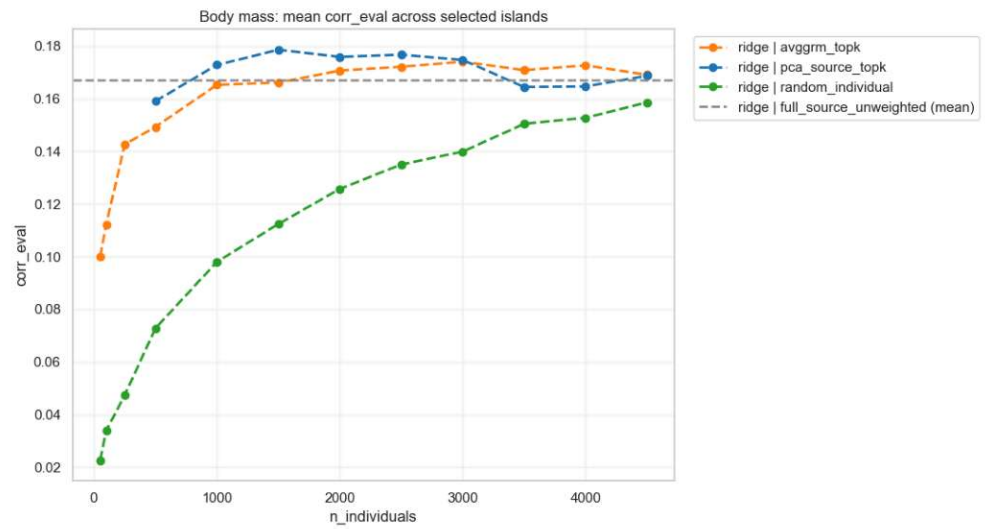
$$\mu_T = \frac{1}{|T|} \sum_{t \in T} z_t$$

- Score each candidate by Euclidean distance to this centroid:

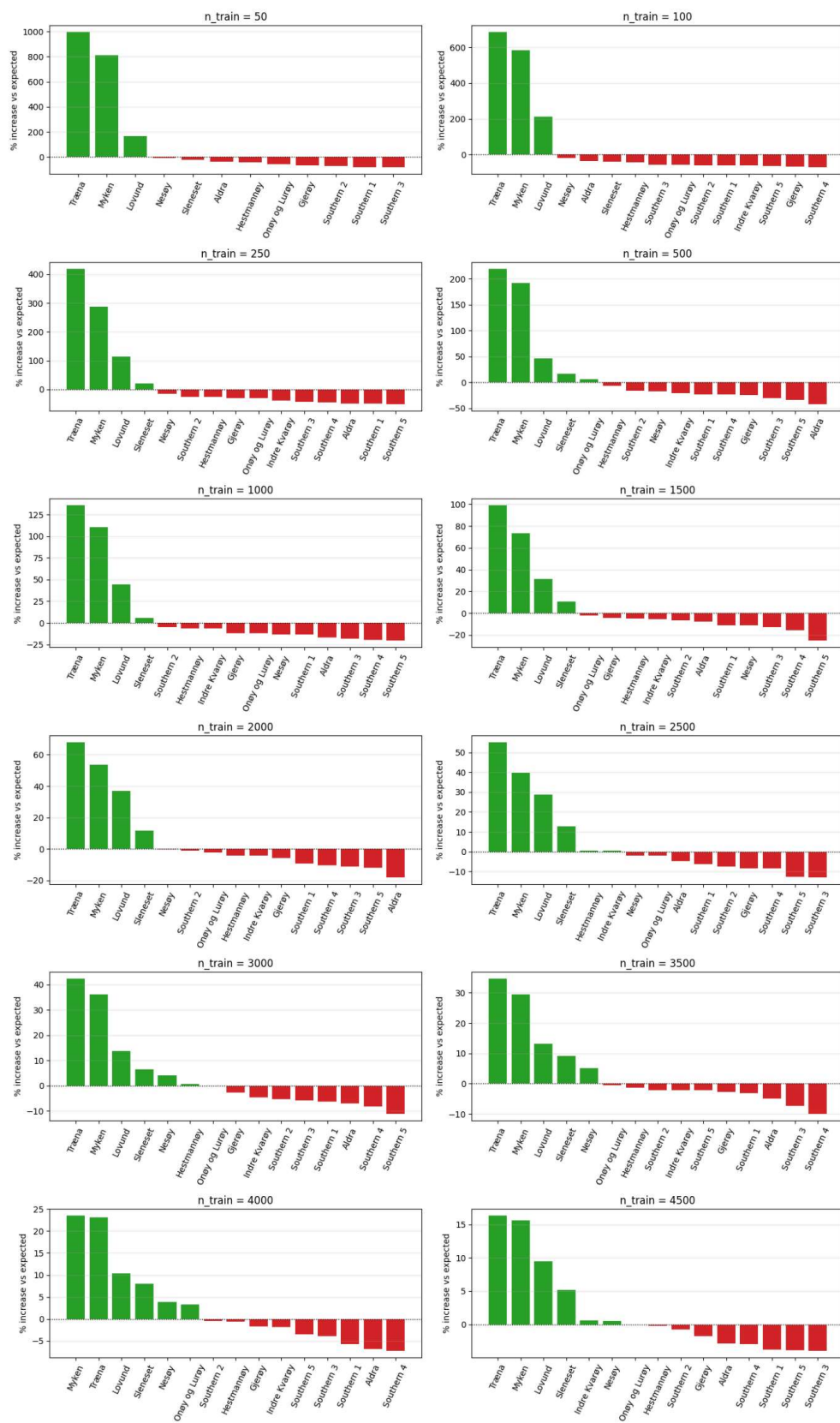
$$d(c) = \|z_c - \mu_T\|_2 = \sqrt{\sum_{j=1}^m (z_{c,j} - \mu_{T,j})^2}$$

- Select top- k candidates with smallest $d(c)$ (nearest neighbors to target island in reduced genetic space).

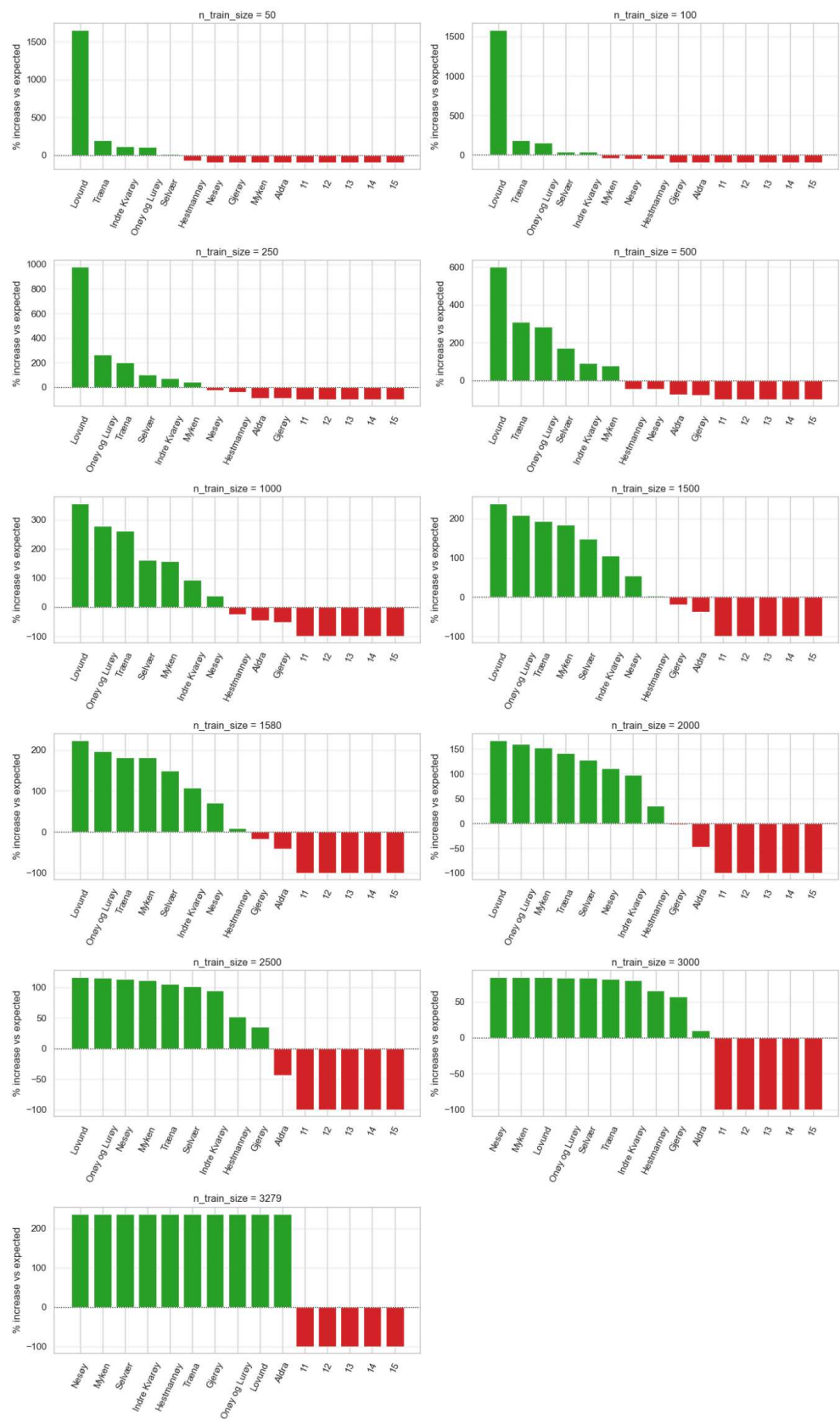
```
In [22]: display_figure(PROJECT_ROOT / "figures" / "avggrm_v_pca.png")
display_figure(PROJECT_ROOT / "figures" / "pev_sel.png")
display_figure(PROJECT_ROOT / "figures" / "avggrm_sel.png")
```



Expected vs actual island contribution per n_train
(sorted by fraction increase over expected)



avggrm_topk: Expected vs actual island contribution for Steneset



Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).